OOP Mid Questions – Answers

1. List out different features of Java. Explain any two.

Features:

- Simple
- Object-Oriented
- Platform-Independent
- Secure
- Robust
- Multithreaded
- Portable
- High Performance

Explain any two:

- **Platform Independent:** Java uses bytecode, which runs on any system with JVM, making it "Write Once, Run Anywhere".
- **Robust:** Java has strong memory management, exception handling, and no

pointer concept, which reduces runtime errors.

---

4. Explain JVM.

JVM (Java Virtual Machine):

- JVM is a virtual machine that runs Java bytecode.
- It converts bytecode into machine code for your system.
- It handles memory allocation, garbage collection, and provides security.
- JVM is platform-dependent but Java code is platform-independent.

5. Compare OOP and POP.

OOP (Object-Oriented)	POP (Procedure-Oriented)
Focuses on objects & data.	Focuses on functions & procedures.
Uses concepts like encapsulation, inheritance, polymorphism.	No data hiding, global data is shared.

OOP (Object-Oriented)	POP (Procedure-Oriented)
Easier to modify & maintain.	Harder to maintain for large projects.
Example: Java, C++	Example: C

---

7. Java Program: Maximum of 10 Numbers using Command-Line Argument

```
class MaxNumber {
    public static void
main(String args[]) {
        int max =
Integer.parseInt(args[0]);
        for (int i = 1; i <
args.length; i++) {
```

```
            int num =
Integer.parseInt(args[i]);
            if (num > max)
                max = num;
        }

System.out.println("Maximum
number = " + max);
    }
}
```

---

9. List out different operators in OOP and explain any two.

Operators in Java:

- Arithmetic (+ - * / %)
- Relational (== != > < >= <=)
- Logical (&& || !)
- Assignment (= += -= *= /=)

- Increment/Decrement (`++ --`)
- Bitwise (`& | ^ ~ << >>`)
- Conditional (`?:`)

Explain any two:

- **Arithmetic Operator:** Used for basic math operations like `a + b`.
- **Relational Operator:** Used for comparison, e.g., `a > b`.

---

10. Explain Wrapper Class with Example.

Wrapper Classes: Convert primitive data types into objects.
Example:

```
public class WrapperExample {
    public static void
main(String[] args) {
        int x = 10;
```

```
        Integer obj =
Integer.valueOf(x); // Boxing
        int y = obj.intValue();
// Unboxing

System.out.println("Value: " +
y);
    }
}
```

---

12. Explain static keyword with example.

- **static** means the member belongs to the class, not to objects.
- Can be used with variables, methods, and blocks.

Example:

```
class Demo {
    static int count = 0;
```

```
    Demo() { count++; }
}
public class Test {
    public static void
main(String[] args) {
        new Demo();
        new Demo();

System.out.println("Objects
created: " + Demo.count);
    }
}
```

---

14. Java Program: Explain String Method

Example using `length()` and `toUpperCase()`:

```
class StringExample {
    public static void
main(String[] args) {
```

```
        String s = "hello
world";

System.out.println("Length: " +
s.length());

System.out.println("Uppercase: "
+ s.toUpperCase());
    }
}
```

---

15. Define Method Overriding with Example.

Method Overriding:
When a subclass provides a new implementation of a method already defined in parent class.

Example:

```
class Animal {
    void sound() {
System.out.println("Animal
sound"); }
}
class Dog extends Animal {
    void sound() {
System.out.println("Dog barks");
}
}
class Test {
    public static void
main(String[] args) {
        Animal a = new Dog();
        a.sound(); // Output:
Dog barks
    }
}
```

---

16. List Types of Inheritance. Explain Multilevel Inheritance.

Types of Inheritance in Java:

- Single
- Multilevel
- Hierarchical
- (Java doesn't support Multiple inheritance with classes)

Multilevel Inheritance Example:

```
class A { void showA() {
System.out.println("A class"); }
}
class B extends A { void showB()
{ System.out.println("B class");
} }
class C extends B { void showC()
{ System.out.println("C class");
} }

public class Test {
    public static void
main(String[] args) {
        C obj = new C();
        obj.showA();
        obj.showB();
```

```
        obj.showC();
    }
}
```

---

18. Explain final keyword with example.

- **final variable:** Value cannot change.
- **final method:** Cannot be overridden.
- **final class:** Cannot be inherited.

Example:

```
final int x = 10; // cannot
reassign x

final class A {}
// class B extends A {}  //
Error - cannot inherit final
class
```

20. Explain abstract keyword with example.

- **abstract class:** Cannot be instantiated, may have abstract (unimplemented) methods.
- **abstract method:** Declared but not implemented in parent class.
-

Example:

```
abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    void sound() {
System.out.println("Bark"); }
}
public class Test {
    public static void
main(String[] args) {
```

```
        Animal a = new Dog();
        a.sound();
    }
}
```
